

第9章 性能调优

性能优化不是一项简单的工作，但也不是复杂的难事，关键在于对 InnoDB 存储引擎特性的了解。如果之前各章的内容读者已经完全理解并掌握了，那就应该基本掌握了如何使 InnoDB 存储引擎更好地工作。本章将从以下几个方面集中讲解 InnoDB 存储引擎的性能问题：

- ☐ 选择合适的 CPU
- ☐ 内存的重要性
- ☐ 硬盘对数据库性能的影响
- ☐ 合理地设置 RAID
- ☐ 操作系统的选择也很重要
- ☐ 不同文件系统对数据库的影响
- ☐ 选择合适的基准测试工具

9.1 选择合适的 CPU

用户首先需要清楚当前数据库的应用类型。一般而言，可分为两大类：OLTP（Online Transaction Processing，在线事务处理）和 OLAP（Online Analytical Processing，在线分析处理）。这是两种截然不同的数据库应用。OLAP 多用在数据仓库或数据集市，一般需要执行复杂的 SQL 语句来进行查询；OLTP 多用在日常的事物处理应用中，如银行交易、在线商品交易、Blog、网络游戏等应用。相对于 OLAP，数据库的容量较小。

InnoDB 存储引擎一般都应用于 OLTP 的数据库应用，这种应用的特点如下：

- ☐ 用户操作的并发量大
- ☐ 事务处理的时间一般比较短
- ☐ 查询的语句较为简单，一般都走索引

❑ 复杂的查询较少

可以看出，OLTP 的数据库应用本身对 CPU 的要求并不是很高，因为复杂的查询可能需要执行比较、排序、连接等非常耗 CPU 的操作，这些操作在 OLTP 的数据库应用中较少发生。因此，可以说 OLAP 是 CPU 密集型的操作，而 OLTP 是 IO 密集型的操作。建议在采购设备时，将更多的注意力放在提高 IO 的配置上。

此外，为了获得更多内存的支持，用户采购的 CPU 必须支持 64 位，否则无法支持 64 位操作系统的安装。因此，为新的应用选择 64 位的 CPU 是必要的前提。现在 4 核的 CPU 已经非常普遍，如今 Intel 和 AMD 又相继推出了 8 核的 CPU，将来随着操作系统的升级我们还可能看到 128 核的 CPU，这都需要数据库更好地对其提供支持。

从 InnoDB 存储引擎的设计架构上来看，其主要的后台操作都是在一个单独的 master thread 中完成的，因此并不能很好地支持多核的应用。当然，开源社区已经通过多种方法来改变这种局面，而 InnoDB 1.0 版本在各种测试下已经显示出对多核 CPU 的处理性能的支持有了极大的提高，而 InnoDB 1.2 版本又支持多个 purge 线程，以及将刷新操作从 master thread 中分离出来。因此，若用户的 CPU 支持多核，InnoDB 的版本应该选择 1.1 或更高版本。另外，如果 CPU 是多核的，可以通过修改参数 innodb_read_io_threads 和 innodb_write_io_threads 来增大 IO 的线程，这样也能更充分有效地利用 CPU 的多核性能。

在当前的 MySQL 数据库版本中，一条 SQL 查询语句只能在一个 CPU 中工作，并不支持多 CPU 的处理。OLTP 的数据库应用操作一般都很简单，因此对 OLTP 应用的影响并不是很大。但是，多个 CPU 或多核 CPU 对处理大并发量的请求还是会有帮助。

9.2 内存的重要性

内存的大小是最能直接反映数据库的性能。通过之前各个章节的介绍，已经了解到 InnoDB 存储引擎既缓存数据，又缓存索引，并且将它们缓存于一个很大的缓冲池中，即 InnoDB Buffer Pool。因此，内存的大小直接影响了数据库的性能。Percona 公司的 CTO Vadim 对此做了一次测试，以此反映内存的重要性，结果如图 9-1 所示。

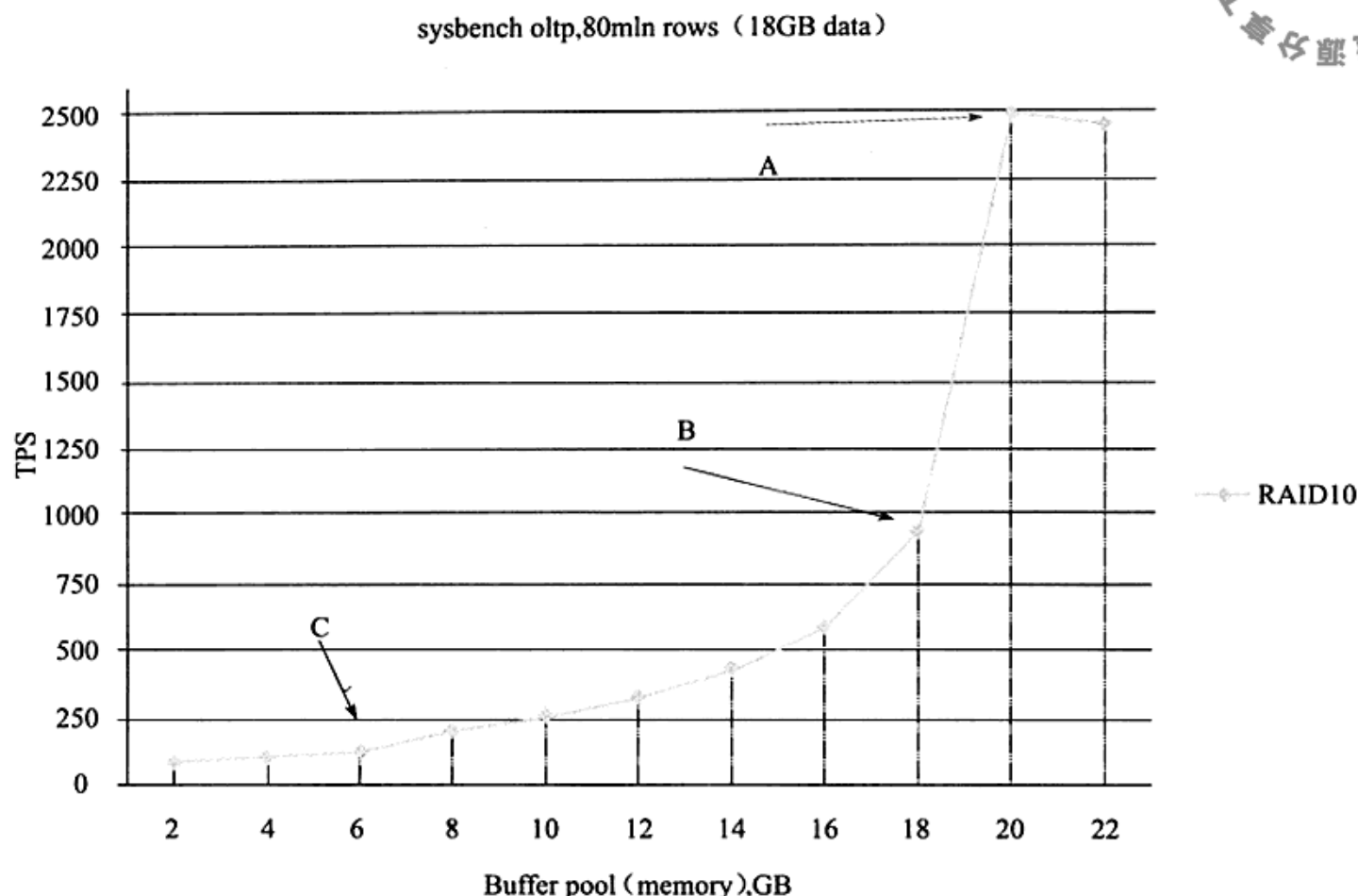


图 9-1 不同内存容量下 InnoDB 存储引擎的性能表现

在上述测试中，数据和索引总大小为 18GB，然后将缓冲池的大小分别设为 2GB、4GB、6GB、8GB、10GB、12GB、14GB、16GB、18GB、20GB、22GB，再进行 sysbench 的测试。可以发现，随着缓冲池的增大，测试结果 TPS (Transaction Per Second) 会线性增长。当缓冲池增大到 20GB 和 22GB 时，数据库的性能有了极大的提高，因为这时缓冲池的大小已经大于数据文件本身的大小，所有对数据文件的操作都可以在内存中进行。因此这时的性能应该是最优的，再调大缓冲池并不能再提高数据库的性能。

所以，应该在开发应用前预估“活跃”数据库的大小是多少，并以此确定数据库服务器内存的大小。当然，要使用更多的内存还必须使用 64 位的操作系统。

如何判断当前数据库的内存是否已经达到瓶颈了呢？可以通过查看当前服务器的状态，比较物理磁盘的读取和内存读取的比例来判断缓冲池的命中率，通常 InnoDB 存储引擎的缓冲池的命中率不应该小于 99%，如：

```
mysql>SHOW GLOBAL STATUS LIKE 'innodb%read%'\G;
***** 1. row *****
Variable_name: Innodb_buffer_pool_read_ahead
Value: 0
```



```

***** 2. row *****
Variable_name: Innodb_buffer_pool_read_ahead_evicted
Value: 0
***** 3. row *****
Variable_name: Innodb_buffer_pool_read_requests
Value: 167051313
***** 4. row *****
Variable_name: Innodb_buffer_pool_reads
Value: 129236
***** 5. row *****
Variable_name: Innodb_data_pending_reads
Value: 0
***** 6. row *****
Variable_name: Innodb_data_read
Value: 2135642112
***** 7. row *****
Variable_name: Innodb_data_reads
Value: 130309
***** 8. row *****
Variable_name: Innodb_pages_read
Value: 130215
***** 9. row *****
Variable_name: Innodb_rows_read
Value: 17651085
9 rows in set (0.00 sec)

```

上述参数的具体含义如表 9-1 所示。

表 9-1 当前服务器的状态参数

参 数	说 明
Innodb_buffer_pool_reads	表示从物理磁盘读取页的次数
Innodb_buffer_pool_read_ahead	预读的页数
Innodb_buffer_pool_read_ahead_evicted	预读的页，但是没有被读取就从缓冲池中被替换的页的数量，一般用来判断预读的效率
Innodb_buffer_pool_read_requests	从缓冲池中读取页的次数
Innodb_data_read	总共读入的字节数
Innodb_data_reads	发起读取请求的次数，每次读取可能需要读取多个页

以下公式可以计算各种对缓冲池的操作：

缓冲池命中率

$$= \frac{\text{Innodb_buffer_pool_read_requests}}{(\text{Innodb_buffer_pool_read_requests} + \text{Innodb_buffer_pool_read_ahead} + \text{Innodb_buffer_pool_reads})}$$

平均每次读取的字节数 = $\frac{\text{Innodb_data_read}}{\text{Innodb_data_reads}}$

从上面的例子看，缓冲池命中率 = $167\,051\,313 / (167\,051\,313 + 129\,236 + 0) = 99.92\%$ 。

即使缓冲池的大小已经大于数据库文件的大小，这也并不意味着没有磁盘操作。数据库的缓冲池只是一个用来存放热点的区域，后台的线程还负责将脏页异步地写入到磁盘。此外，每次事务提交时还需要将日志写入重做日志文件。

9.3 硬盘对数据库性能的影响

9.3.1 传统机械硬盘

当前大多数数据库使用的都是传统的机械硬盘。机械硬盘的技术目前已非常成熟，在服务器领域一般使用 SAS 或 SATA 接口的硬盘。服务器机械硬盘开始向小型化转型，目前大部分使用 2.5 寸的 SAS 机械硬盘。

机械硬盘有两个重要的指标：一个是寻道时间，另一个是转速。当前服务器机械硬盘的寻道时间已经能够达到 3ms，转速为 15 000RPM (rotate per minute)。传统机械硬盘最大的问题在于读写磁头，读写磁头的设计使硬盘可以不再像磁带一样，只能进行顺序访问，而是可以随机访问。但是，机械硬盘的访问需要耗费长时间的磁头旋转和定位来查找，因此顺序访问的速度要远高于随机访问。传统关系数据库的很多设计也都是在尽量充分地利用顺序访问的特性。

通常来说，可以将多块机械硬盘组成 RAID 来提高数据库的性能，也可以将数据文件分布在不同硬盘上来达到访问负载的均衡。

9.3.2 固态硬盘

固态硬盘，更准确地说是基于闪存的固态硬盘，是近几年出现的一种新的存储设备，其内部由闪存 (Flash Memory) 组成。因为闪存的低延迟性、低功耗，以及防震性，闪存设备已在移动设备上得到了广泛的应用。企业级应用一般使用固态硬盘，通过并联多块闪存来进一步提高数据传输的吞吐量。传统的存储服务提供商 EMC 公司已经开始提供基于闪存的固态硬盘的 TB 级别存储解决方案。数据库厂商 Oracle 公司最近也开始提供绑定固态硬盘的 Exadata 服务器。

不同于传统的机械硬盘，闪存是一个完全的电子设备，没有传统机械硬盘的读写磁头。因此，固态硬盘不需要像传统机械硬盘一样，需要耗费大量时间的磁头旋转和定位来查找数据，所以固态硬盘可以提供一致的随机访问时间。固态硬盘这种对数据的快速读写和定位特性是值得研究的。

另一方面，闪存中的数据是不可以更新的，只能通过扇区（sector）的覆盖重写，而在覆盖重写之前，需要执行非常耗时的擦除（erase）操作。擦除操作不能在所含数据的扇区上完成，而需要在删除整个被称为擦除块的基础上完成，这个擦除块的尺寸大于扇区的大小，通常为 128KB 或者 256KB。此外，每个擦除块有擦写次数的限制。已经有一些算法来解决这个问题。但是对于数据库应用，需要认真考虑固态硬盘在写入方面存在的问题。

因为存在上述写入方面的问题，闪存提供的读写速度是非对称的。读取速度要远快于写入的速度，因此对于固态硬盘在数据库中的应用，应该好好利用其读取的性能，避免过多的写入操作。

图 9-2 显示了一个双通道的固态硬盘架构，通过支持 4 路的闪存交叉存储来降低固态硬盘的访问延时，同时增大并发的读写操作。通过进一步增加通道的数量，固态硬盘的性能可以线性地提高，例如我们常见的 Intel X-25M 固态硬盘就是 10 通道的固态硬盘。

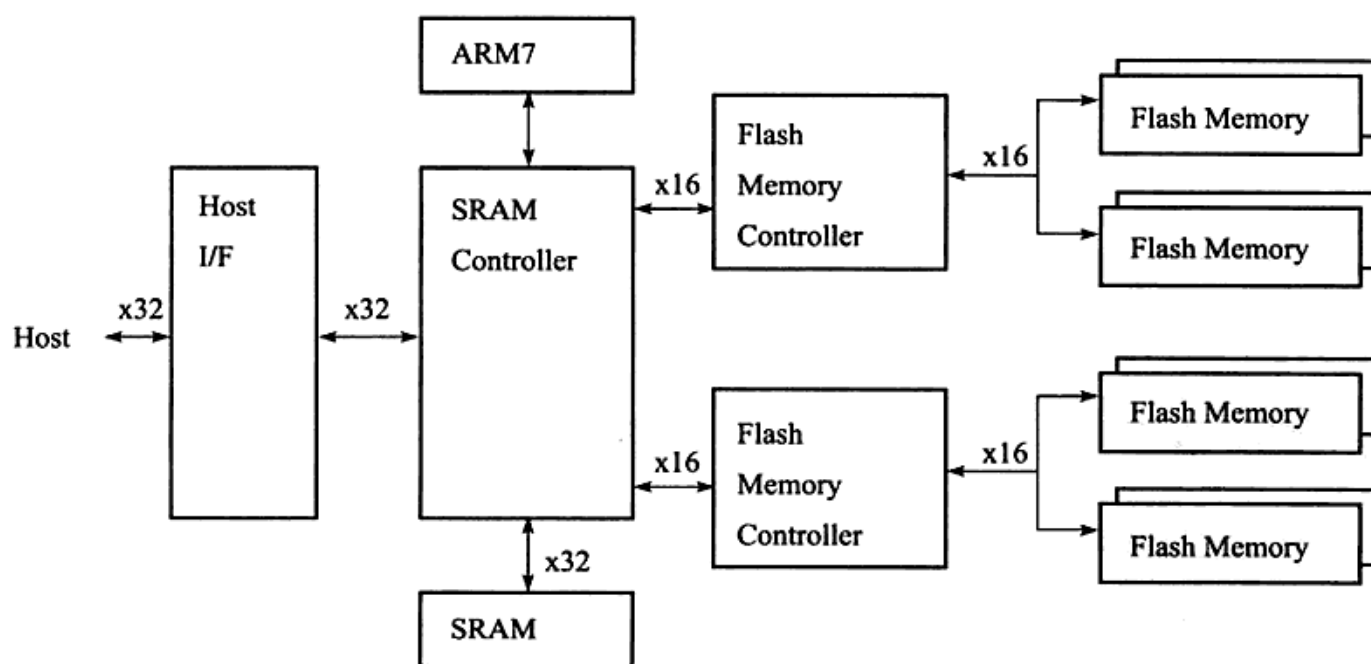


图 9-2 双通道的固态硬盘架构

由于闪存是一个完全的电子设备，没有读写磁头等移动部件，因此固态硬盘有着较低的访问延时。当主机发布一个读写请求时，固态硬盘的控制器会把 I/O 命令从逻辑地址映射成实际的物理地址，写操作还需要修改相应的映射表信息。算上这些额外的开

销, 固态硬盘的访问延时一般小于 0.1ms 左右。图 9-3 显示了传统机械硬盘、内存、固态硬盘的随机访问延时之间的比较。

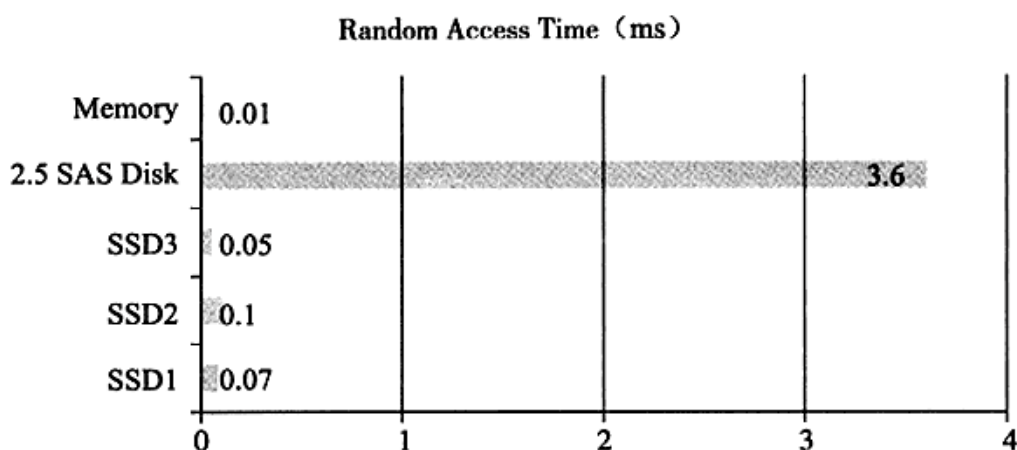


图 9-3 固态硬盘和传统机械硬盘随机访问延时的比较

对于固态硬盘在 InnoDB 存储引擎中的优化, 可以增加 `innodb_io_capacity` 变量的值达到充分利用固态硬盘带来的高 IOPS 特性。不过这需要用户根据自己的应用进行有针对性的调整。在 InnoDB 及 InnoDB1.2 版本中, 可以选择关闭邻接页的刷新, 同样可以为数据库的性能带来一定效果的提升。

此外, 还可以使用 InnoDB 开发的 L2 Cache 解决方案, 该解决方案可以充分利用固态硬盘的超高速随机读取性能, 在内存缓冲池和传统存储层之间建立一层基于闪存固态硬盘的二级缓冲池, 以此来扩充缓冲池的容量, 提高数据库的性能。与基于磁盘的固态硬盘 Cache 类似的解决方案还有 Facebook Flash Cache 和 bcache, 只不过它们是基于通用文件系统的, 对 InnoDB 存储引擎本身的优化较少。

9.4 合理地设置 RAID

9.4.1 RAID 类型

RAID (Redundant Array of Independent Disks, 独立磁盘冗余数组) 的基本思想就是把多个相对便宜的硬盘组合起来, 成为一个磁盘数组, 使性能达到甚至超过一个价格昂贵、容量巨大的硬盘。由于将多个硬盘组合成为一个逻辑扇区, RAID 看起来就像一个单独的硬盘或逻辑存储单元, 因此操作系统只会把它当作一个硬盘。

RAID 的作用是:

□ 增强数据集成度

❑ 增强容错功能

❑ 增加处理量或容量

根据不同磁盘的组合方式，常见的 RAID 组合方式可分为 RAID 0、RAID 1、RAID 5、RAID 10 和 RAID 50 等。

RAID 0：将多个磁盘合并成一个大的磁盘，不会有冗余，并行 I/O，速度最快。RAID 0 亦称为带区集，它将多个磁盘并列起来，使之成为一个大磁盘，如图 9-4 所示。在存放数据时，其将数据按磁盘的个数进行分段，同时将这些数据写进这些盘中。所以，在所有的级别中，RAID 0 的速度是最快的。但是 RAID 0 没有冗余功能，如果一个磁盘（物理）损坏，则所有的数据都会丢失。理论上，多磁盘的效能就等于（单一磁盘效能）×（磁盘数），但实际上受限于总线 I/O 瓶颈及其他因素的影响，RAID 效能会随边递减。也就是说，假设一个磁盘的效能是 50MB/s，两个磁盘的 RAID 0 效能约 96MB/s，三个磁盘的 RAID 0 也许是 130MB/s 而不是 150MB/s。

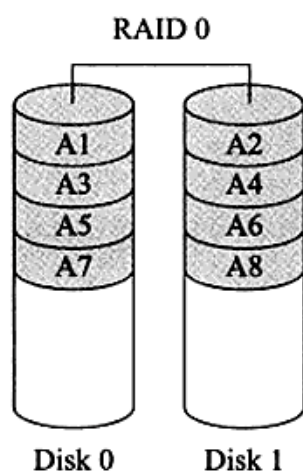


图 9-4 RAID 0 结构

RAID 1：两组以上的 N 个磁盘相互作为镜像（如图 9-5 所示），在一些多线程操作系统中能有很好的读取速度，但写入速度略有降低。除非拥有相同数据的主磁盘与镜像同时损坏，否则只要一个磁盘正常即可维持运作，可靠性最高。RAID 1 就是镜像，其原理为在主硬盘上存放数据的同时也在镜像硬盘上写相同的数据。当主硬盘（物理）损坏时，镜像硬盘则代替主硬盘的工作。因为有镜像硬盘做数据备份，所以 RAID 1 的数据安全性在所有的 RAID 级别上来说是最好的。但是，无论用多少磁盘作为 RAID 1，仅算一个磁盘的容量，是所有 RAID 中磁盘利用率最低的一个级别。

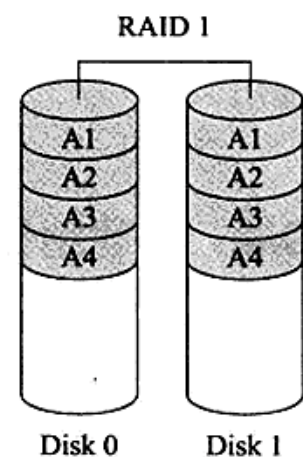


图 9-5 RAID 1 结构

RAID 5：是一种存储性能、数据安全和存储成本兼顾的存储解决方案。它使用的是 Disk Striping（硬盘分区）技术。RAID 5 至少需要三个硬盘，RAID 5 不对存储的数据进行备份，而是把数据和相对应的奇偶校验信息存储到组成 RAID 5 的各个磁盘上，并且奇偶校验信息和相对应的数据分别存储于不同的磁盘上。当 RAID 5 的一个磁盘数据发生损坏后，利用剩下的数据和相应的奇偶校验信息去恢复被损坏的数据。RAID 5 可以理解为是 RAID 0 和 RAID 1 的折中方案。RAID 5 可以为系统提供数据安全保障，但保障程度要比镜像低而磁盘

空间利用率要比镜像高。RAID 5 具有和 RAID 0 相近似的数据读取速度，只是多了一个奇偶校验信息，写入数据的速度相当慢，若使用 Write Back 可以让性能改善不少。同时，由于多个数据对应一个奇偶校验信息，RAID 5 的磁盘空间利用率要比 RAID 1 高，存储成本相对较低。RAID 5 的结构如图 9-6 所示。

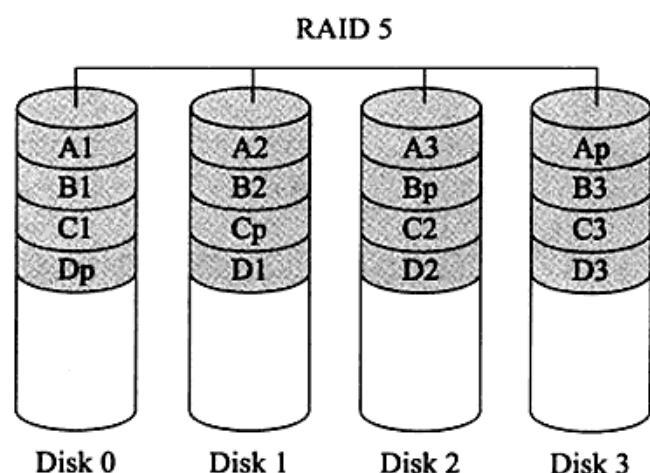


图 9-6 RAID 5 结构

RAID 10 和 RAID 01：RAID 10 是先镜像再分区数据，将所有硬盘分为两组，视为 RAID 0 的最低组合，然后将这两组各自视为 RAID 1 运作。RAID 10 有着不错的读取速度，而且拥有比 RAID 0 更高的数据保护性。RAID 01 则与 RAID 10 的程序相反，先分区再将数据镜像到两组硬盘。RAID 01 将所有的硬盘分为两组，变成 RAID 1 的最低组合，而将两组硬盘各自视为 RAID 0 运作。RAID 01 比 RAID 10 有着更快的读写速度，不过也多了一些会让整个硬盘组停止运转的几率，因为只要同一组的硬盘全部损毁，RAID 01 就会停止运作，而 RAID 10 可以在牺牲 RAID 0 的优势下正常运作。RAID 10 巧妙地利用了 RAID 0 的速度及 RAID 1 的安全（保护）两种特性，它的缺点是需要较多的硬盘，因为至少必须拥有四个以上的偶数硬盘才能使用。RAID 10 和 RAID 01 的结构如图 9-7 所示。

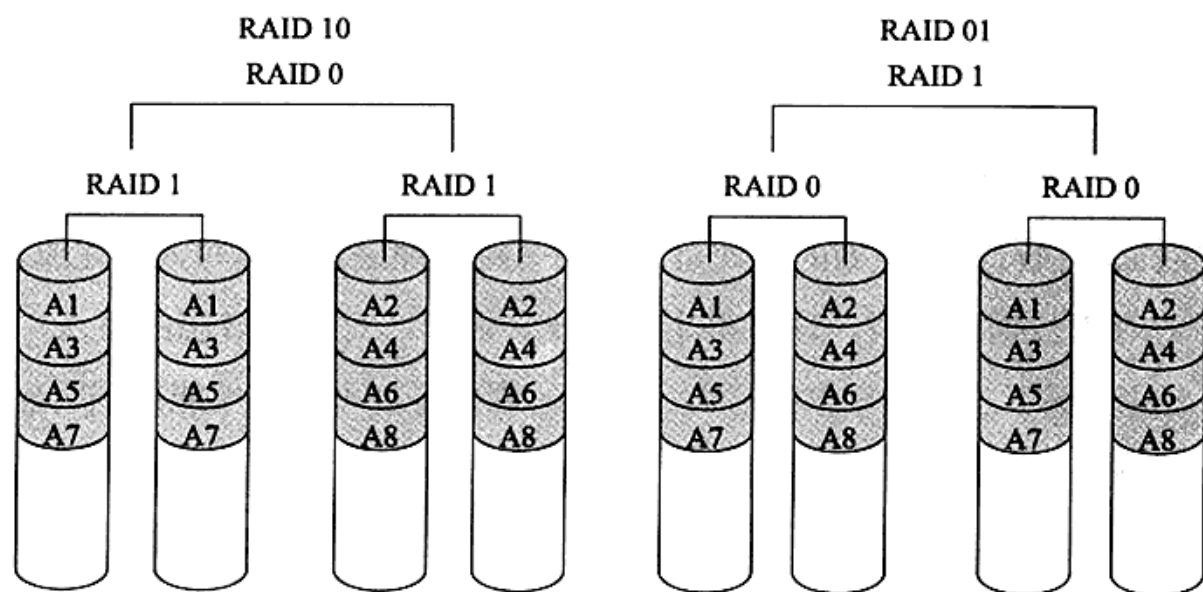


图 9-7 RAID 10 和 RAID 01 结构

RAID 50：RAID 50 也被称为镜像阵列条带，由至少六块硬盘组成，像 RAID 0 一样，数据被分区成条带，在同一时间内向多块磁盘写入；像 RAID 5 一样，也是以数

据的校验位来保证数据的安全，且校验条带均匀分布在各个磁盘上，其目的在于提高 RAID 5 的读写性能。

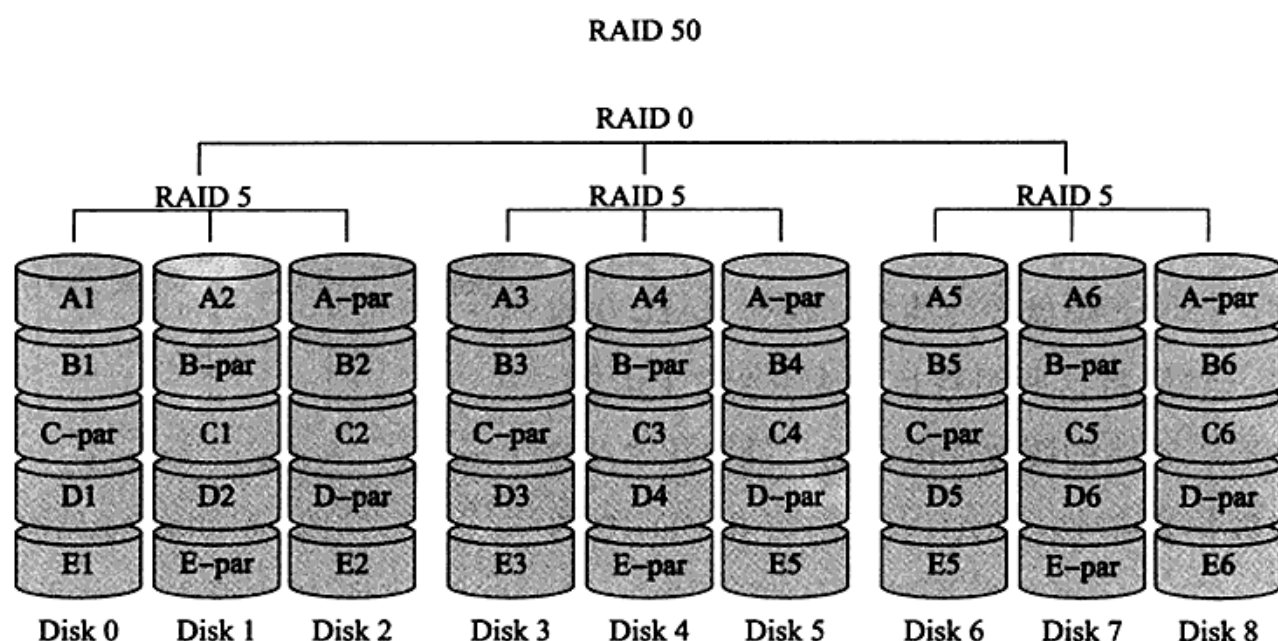


图 9-8 RAID 50 结构

对于数据库应用来说，RAID 10 是最好的选择，它同时兼顾了 RAID 1 和 RAID 0 的特性。但是，当一个磁盘失效时，性能可能会受到很大的影响，因为条带（strip）会成为瓶颈。我曾在生产环境下遇到过的情况是，两台负载基本相同的数据库，一台正常的服务器磁盘 IO 负载为 20% 左右，而另一台服务器 IO 负载却高达 90%。

9.4.2 RAID Write Back 功能

RAID Write Back 功能是指 RAID 控制器能够将写入的数据放入自身的缓存中，并把它们安排到后面再执行。这样做的好处是，不用等待物理磁盘实际写入的完成，因此写入变得更快了。对于数据库来说，这显得十分重要。例如，对重做日志的写入，在将 sync_binlog 设为 1 的情况下二进制日志的写入、脏页的刷新等都可以使性能得到明显的提升。

但是，当操作系统或数据库关机时，Write Back 功能可能会破坏数据库的数据。这是由于已经写入的数据库可能还在 RAID 卡的缓存中，数据可能并没有完全写入磁盘，而这时故障发生了。为了解决这个问题，目前大部分的硬件 RAID 卡都提供了电池备份单元（BBU, Battery Backup Unit），因此可以放心地开启 Write Back 的功能。不过我发现每台服务器的出厂设置都不相同，应该将 RAID 设置要求告知服务器提供商，开启一

些认为需要的参数。

如果没有启用 Write Back 功能，那么在 RAID 卡设置中显示的就是 Write Through。Write Through 没有缓冲写入，因此写入性能可能不是很好，但它却是最安全的写入。

即使用户开启了 Write Back 功能，RAID 卡也可能只是在 Write Through 模式下工作。这是因为安全使用 Write Back 的前提是 RAID 卡有电池备份单元。为了确保电池的有效性，RAID 卡会定期检查电池状态，并在电池电量不足时对其进行充电，在充电的这段时间内会将 Write Back 功能切换为最为安全的 Write Through。

用户可以在没有电池备份单元的情况下强制启用 Write Back 功能，也可以在电池充电时强制使用 Write Back 功能，只是写入是不安全的。用户应该非常确信这点，否则不应该在没有电池备份单元的情况下启用 Write Back。

可以通过插入 20W 的记录来比较 Write Back 和 Write Through 的性能差异：

```
mysql>CREATE TABLE t ( a CHAR(2))Engine=InnoDB;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>DELIMITER //
mysql>
mysql>CREATE PROCEDURE p()
  ->BEGIN
  ->DECLARE v INT;
  ->SET v=0;
  ->WHILE v<200000 DO
  ->INSERTINTO t VALUES('aa');
  ->SET v=v+1;
  ->END WHILE;
  ->END
  -> //
Query OK, 0 rows affected (0.12 sec)
```

```
mysql>DELIMITER ;
```

首先创建一个向表 t 插入 20W 记录的存储过程，并在 Write Back 和 Write Through 的设置下分别进行测试，最终测试结果如表 9-2 所示。

表 9-2 Write Back 和 Write Through 的性能对比测试结果

RAID 卡设置	时 间
Write Back	43 秒
Write Through	31 分钟
Write Through with innodb_flush_log_at_trx_commit=0	68 秒

由于批量插入不是在一个事务中完成的，而是直接用命令 CALL P 来运行的，因此数据库实际执行了 20W 次的事务。很明显可以看到，在 Write Back 模式下执行时间只需要 43 秒，而在 Write Through 模式下执行时间需要 31 分钟，大约有 40 多倍的差距。

当然，在 Write Through 模式下，通过将参数 innodb_flush_log_at_trx_commit 设置为 0 也可以提高执行存储过程 P 的性能，这时只需要 68 秒了。因为，在此设置下，重做日志的写入不是发生在每次事务提交时，而是发生在后台 master 线程每秒钟自动刷新的时候，因此减少了物理磁盘的写入请求，所以执行速度也可以有明显的提高。

9.4.3 RAID 配置工具

对 RAID 卡进行配置可以在服务器启动时进入一个类似于 BIOS 的配置界面，然后再对其进行各种设置。此外，很多厂商都开发了各种操作系统下的软件对 RAID 进行配置，如果用户使用的是 LSI 公司生产提供的 RAID 卡，则可以使用 MegaCLI 工具来进行配置。

MegaCLI 为多个操作系统提供了支持，对 Windows 操作系统还提供了 GUI 界面的配置环境，因此相对来说比较简单。这里主要介绍命令行下 MegaCLI 的使用，在 Windows 下同样可以使用命令 MegaCLI.exe。

使用 MegaCLI 查看 RAID 卡的信息：

```
[root@xen-server ~]# /opt/MegaRAID/MegaCli/MegaCli64 -AdpAllInfo -a0
```

```
Adapter #0
```

```
=====
```

```
Versions
```

```
=====
```

```
Product Name      : MegaRAID SAS 8708ELP
Serial No         : P012233608
FW Package Build  : 9.0.1-0030
```

```
.....
```

```
HW Configuration
```

```
=====
```

```
SAS Address       : 500605b000d1e180
BBU                : Present
Alarm              : Present
```



```

NVRAM           : Present
Serial Debugger : Present
Memory          : Present
Flash           : Present
Memory Size    : 256MB
TPM             : Absent

```

Default Settings

=====

```

Phy Polarity           : 0
PhyPolaritySplit       : 240
Background Rate        : 30
Stripe Size           : 64kB
Flush Time             : 4 seconds
Write Policy        : WB
Read Policy         : None
Cache When BBU Bad  : Disabled
Cached IO           : No
SMART Mode             : Mode 6
Alarm Disable          : Yes
Coercion Mode          : 1GB
ZCR Config             : Unknown
Dirty LED Shows Drive Activity : No
BIOS Continue on Error : No
Spin Down Mode         : None
Allowed Device Type    : SAS/SATA Mix
Allow Mix in Enclosure : Yes
Allow HDD SAS/SATA Mix in VD : Yes
Allow SSD SAS/SATA Mix in VD : No
Allow HDD/SSD Mix in VD : No
Allow SATA in Cluster  : No
Max Chained Enclosures : 3
Disable Ctrl-R         : Yes
Enable Web BIOS        : Yes
Direct PD Mapping      : No
BIOS Enumerate VDs     : Yes
Restore Hot Spare on Insertion : No
Expose Enclosure Devices : Yes
Maintain PD Fail History : Yes
Disable Puncturing     : No
Zero Based Enclosure Enumeration : No
PreBoot CLI Enabled    : Yes
LED Show Drive Activity : No
Cluster Disable        : Yes

```



```

SAS Disable                : No
Auto Detect BackPlane Enable : SGPIO/i2c SEP
Use FDE Only                : No
Enable Led Header           : No
Delay during POST           : 0

```

由于排版的原因，这里只列出了输出的一小部分。通过上述命令可以看到 RAID 卡的一些硬件设置，如这块 RAID 卡的型号是 MegaRAID SAS 8708ELP，缓存大小是 256MB。还可以看到一些默认的配置，如默认启用的 Write Policy 为 WB（Write Back）等。

MegaCLI 还可以用来查看当前物理磁盘的信息，如：

```
[root@xen-server ~]# /opt/MegaRAID/MegaCli/MegaCli64 -PDList -aALL
```

```
Adapter #0
```

```

Enclosure Device ID: 252
Slot Number: 0
Device Id: 8
Sequence Number: 2
Media Error Count: 0
Other Error Count: 0
Predictive Failure Count: 0
Last Predictive Failure Event Seq Number: 0
PD Type: SAS
Raw Size: 279.396 GB [0x22ecb25c Sectors]
Non Coerced Size: 278.896 GB [0x22dcb25c Sectors]
Coerced Size: 278.464 GB [0x22cee000 Sectors]
Firmware state: Online
SAS Address(0): 0x5000c5000f363b55
SAS Address(1): 0x0
Connected Port Number: 0(path0)
Inquiry Data: SEAGATE ST3300655SS      00023LM5MGZZ
FDE Capable: Not Capable
FDE Enable: Disable
Secured: Unsecured
Locked: Unlocked
Foreign State: None
Device Speed: Unknown
Link Speed: Unknown
Media Type: Hard Disk Device
.....

```

可以看到当前使用的磁盘型号是 SEAGATE ST3300655SS。可以从这个型号继续找到

这个硬盘的具体信息，如在希捷官网 <http://discountechnology.com/Seagate-ST3300655SS-SAS-Hard-Drive> 上可以知道这块硬盘大小是 3.5 寸的，转速为 15 000，硬盘的 Cache 为 16MB，随机读取的寻道时间是 3.5 毫秒，随机写入的寻道时间是 4.0 毫秒等。

此外，还可以通过下面的命令来查看是否开启了 Write Back 功能：

```
[root@xen-server ~]# /opt/MegaRAID/MegaCli/MegaCli64 -LDGetProp -Cache -LALL -aALL
```

```
Adapter 0-VD 0(target id: 0): Cache Policy:WriteBack, ReadAheadNone, Direct, No Write Cache if bad BBU
```

```
Adapter 0-VD 1(target id: 1): Cache Policy:WriteBack, ReadAheadNone, Direct, No Write Cache if bad BBU
```

```
Exit Code: 0x00
```

通过上面的结果可以发现当前开启了 RAID 卡的 Write Back 功能，并且当 BBU 有问题时或在充电时禁用 Write Back 功能。此外，这里还显示了不需要启用 RAID 卡的预读功能，写入方式为直接写入。

通过下面的命令可以对当前的写入策略进行调整：

```
#/opt/MegaRAID/MegaCli/MegaCli64 -LDSetPropWB -LALL -aALL
#/opt/MegaRAID/MegaCli/MegaCli64 -LDSetPropWT-LALL -aALL
```

特别需要注意的是，当 RAID 卡的写入策略从 Write Back 切换为 Write Through 时，该更改立即生效。然而从 Write Through 切换为 Write Back 时，必须重启服务器才能使其生效。

9.5 操作系统的选择

Linux 是 MySQL 数据库服务器中最常使用的操作系统。与其他操作系统不同的是 Linux 有着众多的发行版本，每个用户的偏好可能不尽相同。然而在将 Linux 操作系统作为数据库服务器时需要考虑更多的是操作系统的稳定性，而不是新特性。

除了 Linux 操作系统外，FreeBSD 也是另一个常见的优秀操作系统。之前版本的 FreeBSD 对 MySQL 数据库支持得不是很好，需要选择单独的线程库进行手动编译，但是新版本的 FreeBSD 对 MySQL 数据库的支持已经好了很多，直接下载二进制安装包即可。

Solaris 也是非常不错的操作系统，之前是基于 SPARC 硬件的操作系统，现在已经移植到了 X86 平台上。Solaris 是高性能、高可靠性的操作系统，同时其提供的 ZFS 文件系统非常适合 MySQL 的数据库应用。如果需要，用户可以尝试它的开源版本 Open Solaris。

Windows 操作系统在 MySQL 数据库应用中也非常普及。也有公司喜欢在开发环境下使用 Windows 版本的 MySQL 数据库，而在正式生产环境下选择使用 Linux 操作系统。这本身没有什么问题，但问题通常存在于文件系统大小写敏感对应用程序的影响。在 Windows 操作系统下表名不区分大小写，而 Linux 操作系统却是大小写敏感的，这点在开发阶段需要特别注意。

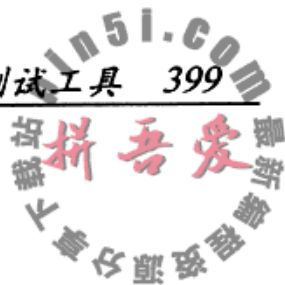
4G 内存在当前已经非常普遍了，即使是桌面用户也开始使用 8G 的内存。为了更好地使用大于 4G 的内存容量，用户必须使用 64 位的操作系统，上述介绍的这些操作系统都提供了 64 位的版本。此外，使用 64 位的操作系统还必须使用 64 位的软件。这听上去像是句废话，但是我曾多次看到 32 位的 MySQL 数据库安装在 64 位的系统上，导致不能充分发挥 64 位操作系统的内存寻址能力。

9.6 不同的文件系统对数据库性能的影响

每个操作系统都默认支持一种文件系统并推荐用户使用，如 Windows 默认支持 NTFS，Solaris 默认支持 ZFS。而对于 Linux 这样的操作系统，不同发行版本默认支持的文件系统各不相同，有的默认支持 EXT3，有的是 ReiserFS，有的是 EXT4，有的是 XFS。

虽然不同特性的文件系统有很多，但是在实际使用过程中从未感觉到文件系统的性能差异有多大。网上有多个关于 XFS 文件系统的“神话”，认为其是多么地适合数据库应用，性能较之 EXT3 有极大的提升。但是在实际测试和使用后发现，它的性能和 EXT3 在整体上没有大的差距。因此，DBA 首先应该把更多的注意力放到数据库上，而不是纠结于文件系统。

文件系统可提供的功能也许是 DBA 需要关注的，例如 ZFS 文件系统本身就可以支持快照，因此就不需要 LVM 这样的逻辑卷管理工具。此外，可能还需要知道 mount 的参数，这些参数在每个文件系统中可能有所不同。



9.7 选择合适的基准测试工具

基准测试工具可以用来对数据库或操作系统调优后的性能进行对比。MySQL 数据库本身提供了一些比较优秀的工具，这里将介绍另外两款更为优秀和常用的基准测试工具：sysbench 和 mysql-tpcc。

9.7.1 sysbench

sysbench 是一个模块化的、跨平台的多线程基准测试工具，主要用于测试各种不同系统参数下的数据库负载情况。它主要包括以下几种测试方式：

- ☐ CPU 性能
- ☐ 磁盘 IO 性能
- ☐ 调度程序性能
- ☐ 内存分配及传输速度
- ☐ POSIX 线程性能
- ☐ 数据库 OLTP 基准测试

sysbench 的数据库 OLTP 测试支持 MySQL、PostgreSQL 和 Oracle。目前 sysbench 主要用于 Linux 操作系统，开源社区已经将 sysbench 移植到 Windows，并支持对 Microsoft SQL Server 数据库的测试。

sysbench 的官网地址是：<http://sysbench.sourceforge.net>，可以从该地址下载最新版本的 sysbench 工具，然后进行编译和安装。此外，有些 Linux 操作系统发行版本，如 RED HAT，本身可能已经提供了 sysbench 的安装包，直接安装即可。

sysbench 可以通过不同的参数设置来进行不同项目的测试，使用方法如下：

```
[root@xen-server ~]# sysbench
Missing required command argument.
Usage:
  sysbench [general-options]... --test=<test-name> [test-options]... command

General options:
  --num-threads=N           number of threads to use [1]
  --max-requests=N          limit for total number of requests [10000]
  --max-time=N              limit for total execution time in seconds [0]
  --thread-stack-size=SIZE  size of stack per thread [32K]
```

```
--init-rng=[on|off]      initialize random number generator [off]
--test=STRING            test to run
--debug=[on|off]         print more debugging info [off]
--validate=[on|off]      perform validation checks where possible [off]
--help=[on|off]          print help and exit
--version=[on|off]       print version and exit
```

Compiled-in tests:

```
fileio - File I/O test
cpu - CPU performance test
memory - Memory functions speed test
threads - Threads subsystem performance test
mutex - Mutex performance test
oltp - OLTP test
```

Commands: prepare run cleanup help version

See 'sysbench --test=<name> help' for a list of options for each test.

对于 InnoDB 存储引擎的数据库应用来说，用户可能更关心磁盘和 OLTP 的性能，因此主要测试 fileio 和 oltp 这两个项目。对于磁盘的测试，sysbench 提供了以下的测试选项：

```
[root@xen-server ~]# sysbench --test=fileio help
```

```
sysbench 0.4.10: multi-threaded system evaluation benchmark
```

fileio options:

```
--file-num=N              number of files to create [128]
--file-block-size=N       block size to use in all IO operations [16384]
--file-total-size=SIZE    total size of files to create [2G]
--file-test-mode=STRING   test mode {seqwr, seqrewr, seqrd, rndrd, rndwr,
rndrw}
--file-io-mode=STRING     file operations mode {sync,async,fastmmap,slowmmap}
[sync]
--file-extra-flags=STRING additional flags to use on opening files
{sync,dsync,direct} []
--file-fsync-freq=N       do fsync() after this number of requests (0 -
don't use fsync()) [100]
--file-fsync-all=[on|off] do fsync() after each write operation [off]
--file-fsync-end=[on|off] do fsync() at the end of test [on]
--file-fsync-mode=STRING  which method to use for synchronization {fsync,
fdatasync} [fsync]
--file-merged-requests=N  merge at most this number of IO requests if
possible (0 - don't merge) [0]
--file-rw-ratio=N         reads/writes ratio for combined test [1.5]
```

各个参数的含义如下：

- ☐ `--file-num`, 生成测试文件的数量, 默认为 128。
- ☐ `--file-block-size`, 测试期间文件块的大小, 如果想知道磁盘针对 InnoDB 存储引擎进行的测试, 可以将其设置为 16384, 即 InnoDB 存储引擎页的大小。默认为 16384。
- ☐ `--file-total-size`, 每个文件的大小, 默认为 2GB。
- ☐ `--file-test-mode`, 文件测试模式, 包含 `seqwr` (顺序写)、`seqrewr` (顺序读写)、`seqrd` (顺序读)、`rndrd` (随机读)、`rndwr` (随机写) 和 `rndrw` (随机读写)。
- ☐ `--file-io-mode`, 文件操作的模式, 同步还是异步, 或者是选择 `MMAP` (map 映射) 模式。默认为同步。
- ☐ `--file-extra-flags`, 打开文件时的选项, 这是与 API 相关的参数。
- ☐ `--file-fsync-freq`, 执行 `fsync` 函数的频率。`fsync` 主要是同步磁盘文件, 因为可能有系统和磁盘缓冲的关系。
- ☐ `--file-fsync-all`, 每执行完一次写操作, 就执行一次 `fsync`。默认为 `off`。
- ☐ `--file-fsync-end`, 在测试结束时, 执行 `fsync`。默认为 `on`。
- ☐ `--file-fsync-mode`, 文件同步函数的选择, 同样是和 API 相关的参数, 由于多个操作系统对 `fdatsync` 支持的不同, 因此不建议使用 `fdatsync`。默认为 `fsync`。
- ☐ `--file-rw-ratio`, 测试时的读写比例, 默认是 2 : 1。

`sysbench` 的 `fileio` 测试需要经过 `prepare`、`run` 和 `cleanup` 三个阶段。`prepare` 是准备阶段, 生产需要的测试文件, `run` 是实际测试阶段, `cleanup` 是清理测试产生的文件。例如进行 16 个文件、总大小 2GB 的 `fileio` 测试:

```
[root@xen-server ssd]# sysbench --test=fileio --file-num=16 --file-total-size=2G
prepare
sysbench 0.4.10: multi-threaded system evaluation benchmark

16 files, 131072Kb each, 2048Mb total
Creating files for the test...
```

接着在相应的目录下就会产生 16 个文件, 因为总大小是 2GB, 所以每个文件的大小应该是 128MB。

```
[root@xen-server ssd]# ls -lh
total 2G
```



```
-rw----- 1 root  root  128M Aug 12 10:42 test_file.0
-rw----- 1 root  root  128M Aug 12 10:42 test_file.1
-rw----- 1 root  root  128M Aug 12 10:42 test_file.10
-rw----- 1 root  root  128M Aug 12 10:42 test_file.11
-rw----- 1 root  root  128M Aug 12 10:42 test_file.12
-rw----- 1 root  root  128M Aug 12 10:42 test_file.13
-rw----- 1 root  root  128M Aug 12 10:42 test_file.14
-rw----- 1 root  root  128M Aug 12 10:42 test_file.15
-rw----- 1 root  root  128M Aug 12 10:42 test_file.2
-rw----- 1 root  root  128M Aug 12 10:42 test_file.3
-rw----- 1 root  root  128M Aug 12 10:42 test_file.4
-rw----- 1 root  root  128M Aug 12 10:42 test_file.5
-rw----- 1 root  root  128M Aug 12 10:42 test_file.6
-rw----- 1 root  root  128M Aug 12 10:42 test_file.7
-rw----- 1 root  root  128M Aug 12 10:42 test_file.8
-rw----- 1 root  root  128M Aug 12 10:42 test_file.9
```

接着就可以基于这些文件进行测试了。下面是在 16 个线程下的随机读取性能：

```
[root@xen-server ssd]# sysbench --test=fileio --file-total-size=2G --file-test-
mode=rndrd --max-time=180 --max-requests=100000000 --num-threads=16 --init-rng=on
--file-num=16 --file-extra-flags=direct --file-fsync-freq=0 --file-block-size=16384 run
```

上述测试的最大随机读取请求是 100 000 000 次，如果在 180 秒内不能完成，测试即结束。测试结束后可以看到如下的测试结果：

```
[root@xen-server ssd]# sysbench --test=fileio --file-total-size=2G --file-test-
mode=rndrd --max-time=180 --max-requests=100000000 --num-threads=16 --init-rng=on
--file-num=16 --file-extra-flags=direct --file-fsync-freq=0 --file-block-size=16384 run
sysbench 0.4.10: multi-threaded system evaluation benchmark
```

Running the test with following options:

Number of threads: 16

Initializing random number generator from timer.

Extra file open flags: 16384

16 files, 128Mb each

2Gb total file size

Block size 16Kb

Number of random requests for random IO: 100000000

Read/Write ratio for combined random IO test: 1.50

Calling fsync() at the end of test, Enabled.

Using synchronous I/O mode

Doing random read test

Threads started!



```
Time limit exceeded, exiting...
(last message repeated 15 times)
Done.
```

```
Operations performed: 619908 Read, 0 Write, 0 Other = 619908 Total
Read 9.459Gb Written 0b Total transferred 9.459Gb (53.81Mb/sec)
3443.85 Requests/sec executed
```

Test execution summary:

```
total time: 180.0044s
total number of events: 619908
total time taken by event execution: 2878.0750
per-request statistics:
  min: 0.42ms
  avg: 4.64ms
  max: 27.30ms
  approx. 95 percentile: 8.13ms
```

Threads fairness:

```
events (avg/stddev): 38744.2500/102.69
execution time (avg/stddev): 179.8797/0.00
```

可以看到随机读取的性能为 53.81MB/s，随机读的 IOPS 为 3443.85。测试的硬盘是固态硬盘，因此随机读取的性能较为强劲。此外还可以看到每次请求的一些具体数据，如最大值、最小值、平均值等。

测试结束后，记得要执行 `cleanup`，确保测试产生的文件都已删除：

```
[root@xen-server ssd]# sysbench --test=fileio --file-num=16 --file-total-size=2G
cleanup
```

```
sysbench 0.4.10: multi-threaded system evaluation benchmark
```

```
Removing test files...
```

可能用户需要测试随机读、随机写、随机读写、顺序写、顺序读等所有这些模式，并且还可能测试不同的线程和不同文件块下磁盘的性能表现，这时可能需要类似如下的脚本来帮用户自动完成这些测试：

```
#!/bin/sh
set -u
set -x
set -e
for size in 8G 64G; do
for mode in seqrd seqrw rndrd rndwr rndrw; do
for blksize in 4096 16384 ; do
```



```

sysbench --test=fileio --file-num=64 --file-total-size=$size prepare
for threads in 1 4 8 16 32; do
echo "==== testing $blksize in $threads threads"
echo PARAMS $size $mode $threads $blksize>sysbench-size-$size-mode-$mode-
threads-$threads-blksz-$blksize
for i in 1 2 3 ; do
sysbench --test=fileio --file-total-size=$size --file-test-mode=$mode\
--max-time=180 --max-requests=100000000 --num-threads=$threads --init-rng=on \
--file-num=64 --file-extra-flags=direct --file-fsync-freq=0 --file-block-
size=$blksize run \
| tee -a sysbench-size-$size-mode-$mode-threads-$threads-blksz-$blksize 2>&1
done
done
sysbench --test=fileio --file-total-size=$size cleanup
done
done
done

```

对于 MySQL 数据库的 OLTP 测试，和 fileio 一样需要经历 prepare、run 和 cleanup 阶段。prepare 阶段会根据选项产生一张指定行数的表，默认表在 sbtest 架构下，表名为 sbtest（sysbench 默认生成表的存储引擎为 InnoDB）。例如创建一张 8000W 的表：

```

[root@xen-server ~]# sysbench --test=oltp --oltp-table-size= 80000000 --db-
driver=mysql --mysql-socket=/tmp/mysql.sock --mysql-user=root prepare
sysbench 0.4.10: multi-threaded system evaluation benchmark

Creating table 'sbtest'...
Creating 80000000 records in table 'sbtest'...

```

接着就可以根据产生的表进行 oltp 的测试：

```

sysbench --test=oltp --oltp-table-size=80000000 --oltp-read-only=off --init-
rng=on --num-threads=16 --max-requests=0 --oltp-dist-type=uniform --max-time=3600
--mysql-user=root --mysql-socket=/tmp/mysql.sock --db-driver=mysql run > res

```

用户可将测试结果放入到了文件 res 中，查看 res 可得类似如下结果：

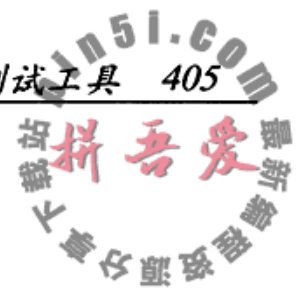
```

sysbench 0.4.10: multi-threaded system evaluation benchmark

WARNING: Preparing of "BEGIN" is unsupported, using emulation
(last message repeated 15 times)
Running the test with following options:
Number of threads: 16
Initializing random number generator from timer.

Doing OLTP test.

```

```
Running mixed OLTP test
Using Uniform distribution
Using "BEGIN" for starting transactions
Using auto_inc on the id column
Threads started!
Time limit exceeded, exiting...
(last message repeated 15 times)
Done.
```

OLTP test statistics:

```
queries performed:
  read:                      6043324
  write:                     2158330
  other:                      863332
  total:                     9064986
transactions:                431666 (119.90 per sec.)
deadlocks:                   0      (0.00 per sec.)
read/write requests:         8201654 (2278.07 per sec.)
other operations:            863332 (239.80 per sec.)
```

Test execution summary:

```
total time:                  3600.2672s
total number of events:      431666
total time taken by event execution: 57598.5965
per-request statistics:
  min:                        6.84ms
  avg:                        133.43ms
  max:                        7155.61ms
  approx. 95 percentile:     325.84ms
```

Threads fairness:

```
events (avg/stddev):        26979.1250/64.14
execution time (avg/stddev): 3599.9123/0.06.
```

结果中罗列出了测试时很多操作的详细信息，transactions 代表了测试结果的评判标准，即 TPS，上述测试的结果是 119.9tps。用户可以对数据库进行调优后再运行 sysbench 的 OLTP 测试，看看 TPS 是否有所提高。注意，sysbench 的测试只是基准测试，并不代表实际生产环境下的性能指标。

9.7.2 mysql-tpcc

TPC (Transaction Processing Performance Council, 事务处理性能协会) 是一个用来

评价大型数据库系统软硬件性能的非盈利组织。TPC-C 是 TPC 协会制定的,用来测试典型的复杂 OLTP (在线事务处理) 系统的性能。目前在学术界和工业界普遍采用 TPC-C 来评价 OLTP 应用的性能。

TPC-C 用 3NF (第三范式) 虚拟实现了一家仓库销售供应商公司,拥有一批分布在不同地方的仓库和地区分公司。当公司业务扩大时,将建立新的仓库和地区分公司。通常每个仓库供货覆盖 10 家地区分公司,每个地区分公司服务 3000 名客户。公司共有 100 000 种商品,分别储存在各个仓库中。该系统包含了库存管理、销售、分发产品、付款、订单查询等一系列操作,一共包含了 9 个基本关系,基本关系如图 9-9 所示。

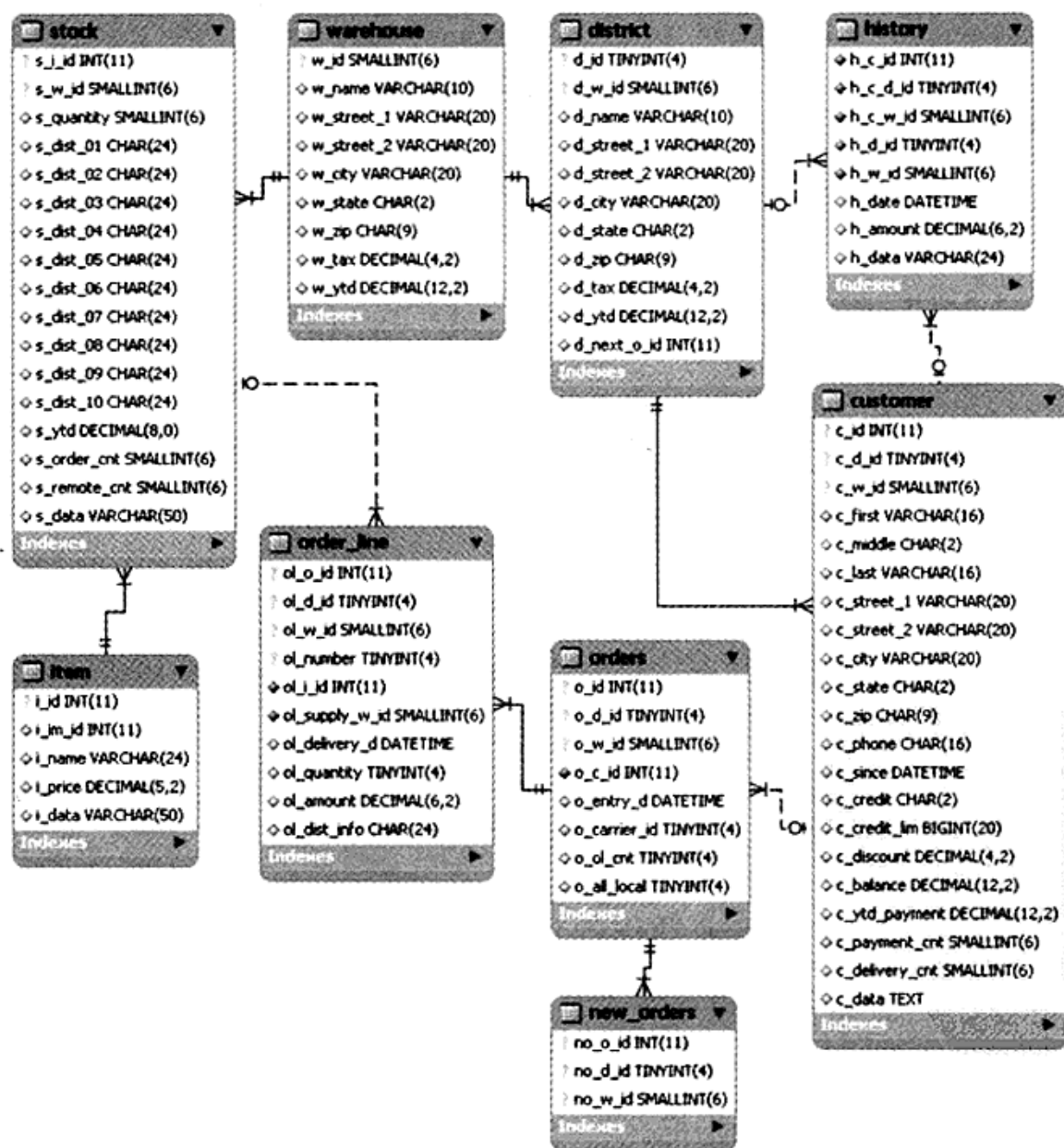


图 9-9 TPC-C 基本关系图

TPC-C 的性能度量单位是 tpmC, tpm 是 transaction per minute 的缩写, C 代表 TPC 的 C 基准测试。该值越大,代表事务处理的性能越高。

tpcc-mysql 是开源的 TPC-C 测试工具，该测试工具完全遵守 TPC-C 的标准。其官方网站为：<https://code.launchpad.net/~percona-dev/perconatools/tpcc-mysql>。之前 tpcc-mysql 主要工作在 Linux 操作系统上，我已经将其移植到了 Windows 平台，可以在 <http://code.google.com/p/david-mysql-tools/downloads/list> 下载到 Windows 版本的 tpcc-mysql。

tpcc-mysql 由以下两个工具组成。

☐ tpcc_load: 根据仓库数量，生成 9 张表中的数据。

☐ tpcc_start: 根据不同选项进行 TPC-C 测试。

tpcc_load 命令的使用方法如下：

```
[root@xen-server ~]# tpcc_load
*****
*** ###easy### TPC-C Data Loader ***
*****

usage: tpcc_load [server] [DB] [user] [pass] [warehouse]
      OR
      tpcc_load [server] [DB] [user] [pass] [warehouse] [part] [min_wh] [max_wh]

      * [part]: 1=ITEMS 2=WAREHOUSE 3=CUSTOMER 4=ORDERS
```

各参数的意义如下：

☐ server, 导入的 MySQL 服务器 IP。

☐ DB, 导入的数据库。

☐ user, MySQL 的用户名。

☐ pass, MySQL 的密码。

☐ warehouse, 要生产的仓库数量。

如果用 tpcc_load 工具创建 100 个仓库的数据库 tpcc, 可以这样：

```
[root@xen-server tpcc-mysql]# mysql tpcc<create_table.sql
[root@xen-server tpcc-mysql]# mysql tpcc<add_fkey_idx.sql
[root@xen-server tpcc-mysql]# tpcc_load 127.0.0.1 tpcc2 root xxxxxx 100
*****
*** ###easy### TPC-C Data Loader ***
*****
<Parameters>
  [server]: 127.0.0.1
```



```

[DBname]: tpcc2
[user]: root
[pass]:
[warehouse]: 100
TPCC Data Load Started...
Loading Item
..... 5000
..... 10000
..... 15000
..... (略)
...DATA LOADING COMPLETED SUCCESSFULLY.

```

tpcc_start 命令的使用方法如下:

```

[root@xen-server ~]# tpcc_start
*****
*** ###easy### TPC-C Load Generator ***
*****

```

```

usage: tpcc_start [server] [DB] [user] [pass] [warehouse] [connection] [rampup]
[measure]

```

相关参数的作用如下:

- ☐ connection, 测试时的线程数量。
- ☐ rampup, 热身时间, 单位秒, 这段时间的操作不计入统计信息。
- ☐ measure, 测试时间, 单位秒。

如使用 tpcc_start 进行 16 个线程的测试, 热身时间为 10 分钟, 测试时间为 20 分钟, 如下:

```

[root@xen-server ~]# tpcc_start 127.0.0.1 tpcc root xxxxxx 100 16 600 1200
*****
*** ###easy### TPC-C Load Generator ***
*****
<Parameters>
  [server]: 127.0.0.1
  [DBname]: tpcc
  [user]: root
  [pass]: xxxxxx
  [warehouse]: 100
  [connection]: 16
  [rampup]: 600 (sec.)
  [measure]: 1200 (sec.)
.....

```

在测试的时候用户或许会在终端上看到类似如下的输出：

RAMP-UP TIME.(1 sec.)

MEASURING START.

```
10, 624(0):0.4, 624(0):0.2, 62(0):0.2, 63(0):0.6, 62(0):0.8
20, 990(0):0.2, 988(0):0.2, 98(0):0.2, 99(0):0.4, 98(0):0.6
30, 1435(0):0.2, 1436(0):0.2, 144(0):0.2, 143(0):0.2, 144(0):0.4
40, 1736(0):0.2, 1739(0):0.2, 174(0):0.2, 174(0):0.2, 174(0):0.4
50, 2041(0):0.2, 2044(0):0.2, 204(0):0.2, 204(0):0.2, 207(0):0.2
60, 2195(0):0.2, 2193(0):0.2, 220(0):0.2, 221(0):0.2, 218(0):0.2
70, 2332(0):0.2, 2335(0):0.2, 233(0):0.2, 232(0):0.2, 234(0):0.2
80, 2408(0):0.2, 2401(0):0.2, 241(0):0.2, 239(0):0.2, 241(0):0.2
90, 2473(0):0.2, 2476(0):0.2, 247(0):0.2, 250(0):0.2, 248(0):0.2
100, 2350(0):0.2, 2347(0):0.2, 235(0):0.2, 233(0):0.2, 235(0):0.2
.....
```

这些信息是每 10 秒 TPC-C 测试的结果数据，TPC-C 测试一共测试 5 个模块，分别是 New Order、Payment、Order-Status、Delivery、Stock-Level。第一个值即为 New Order，这也是 TPC-C 测试结果的一个重要考量标准 New Order Per 10 Second（每十秒订单处理能力），可以将测试时所有的数据组成一张折线图或散点图，观察 InnoDB 存储引擎每 10 秒的性能表现，如图 9-10 所示。

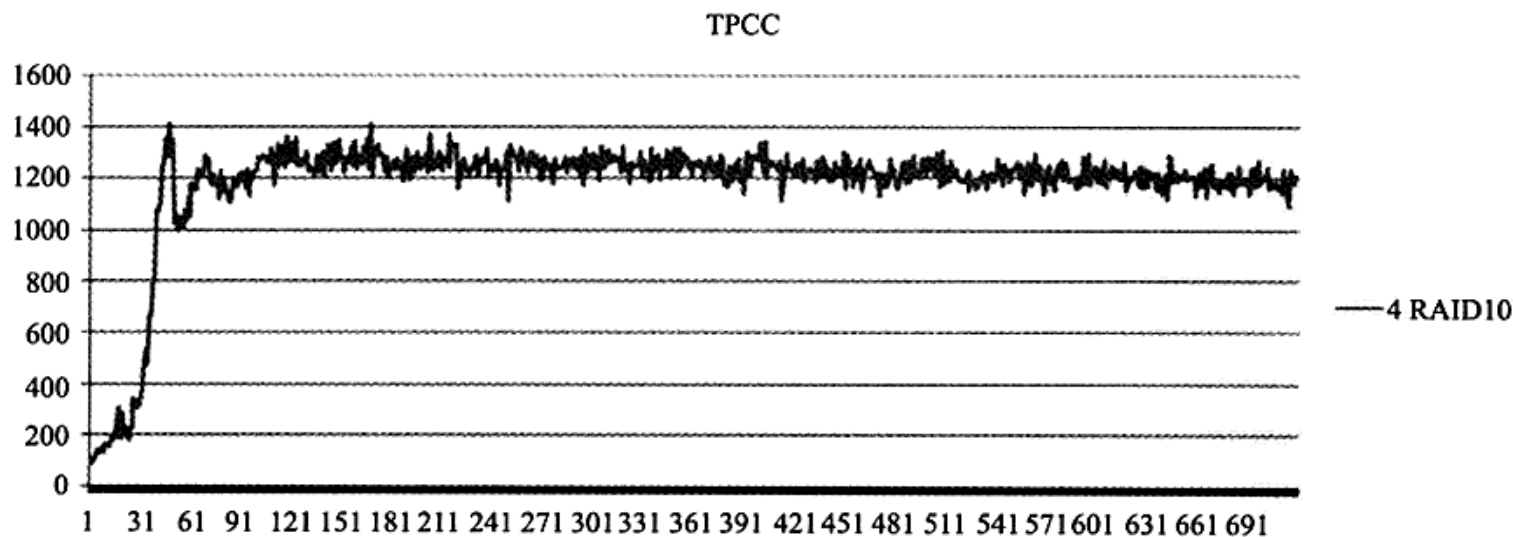


图 9-10 New Order Per 10 Second

而 tpcc_load 最后结束时产生的 tpmC 也是通过 New Order Per 10 Second 来进行的，首先求出 New Order Per 10 Second 的平均值，然后乘以 6，得到的就是最终的 tpmC。

.....

```
<Constraint Check> (all must be [OK])
[transaction percentage]
    Payment: 43.48% (>=43.0%) [OK]
    Order-Status: 4.35% (>= 4.0%) [OK]
    Delivery: 4.35% (>= 4.0%) [OK]
    Stock-Level: 4.35% (>= 4.0%) [OK]
[response time (at least 90% passed)]
    New-Order: 99.72% [OK]
    Payment: 99.95% [OK]
    Order-Status: 99.93% [OK]
    Delivery: 100.00% [OK]
    Stock-Level: 100.00% [OK]
```

<TpmC>

7949.942 TpmC

9.8 小结

在这一章中我们根据 InnoDB 存储引擎的应用特点对 CPU、内存、硬盘、固态硬盘、RAID 卡做了详细的介绍，相信只有通过理解 InnoDB 存储引擎的应用场合和范围才能更好地对其进行调优。最后，介绍了两个在 Linux 操作系统平台下常用的基准测试工具 sysbench 和 tpcc-mysql，借助这两个工具可以更有效地得知当前系统的负载承受能力，以及对 MySQL 数据库的调优结果进行分析。